The Winner-Take-All Partition Gap at High N

Phase 1 satisfies the *continuous* equal-area constraint but *manufactures* a grossly undersized "runt" cell during relaxation, which makes Phase 2 perimeter refinement infeasible — located to the refinement level at which it happens, on three torus runs

July 2026

Status — measured: the failure is reproduced and its origin located across all five refinement levels of three runs

This report **characterises** the high-N failure whose standing explanation lives in `docs/reference/winner_take_all_partition_gap.md` and whose downstream cost gates `docs/plans/PHASE1_N1000_SCALING_PLAN.md`. The numbers come from re-reading three completed Phase 1 runs (*no new optimisation*); the reconstruction is anchored on a value it must reproduce (the Phase 2 iteration-0 constraint violation). It measures *where* the runt is born; it does not implement or evaluate a fix (candidates are listed in §5 and ranked in the reference doc).

Provenance

Date	2026-07-06 (analysis of runs completed 2026-06-25 / 06-29 / 07-01)
Surface	torus $T(R{=}1.0,\ r{=}0.6)$, $\lambda_{\text{penalty}}{=}2.1$, seed 84172851, seeded init, 5 levels
Source runs	N=50 `run_20260625_113015` (finest 348×328, V=114,144); N=100 coarse `run_20260629_141012` (same finest mesh); N=100 finer `run_20260701_143238` (finest 494×464, V=229,216)
Producing script	`make_figures.py` (beside this report; run from repo root)
Method inputs	per-level density snapshots from each run's `traces/`; meshes rebuilt with `src/surfaces/torus.py`
Versions	Python 3.12.13, numpy 2.4.4, scipy 1.17.1, matplotlib 3.10.8, h5py 3.16.0
Anchor	finer-run cell 92 winner-take-all area = 0.0774, worst-cell abs. deviation = 0.160 — matching the Phase 2 iteration-0 constraint violation 1.60e−1 in that run's `refinement.log`

Abstract

Status: measured. Phase 1 minimises a Γ-convergence energy over *continuous density functions* and enforces equal areas on them; the actual partition is then extracted by a *winner-take-all* (argmax) hard decision. These two representations disagree at high N: a cell can hold its full continuous mass $\int u_k\,dA = A/N$ exactly while its discrete winner-take-all territory is a fraction of A/N. On the torus at $N = 100$, two runs failed Phase 2 perimeter refinement — the perimeter *rose* every iteration and the solver stopped at a point of local infeasibility — because one such "runt" cell makes the Phase 2 equal-area constraint infeasible from iteration 0. Tracking every cell's territory across all five refinement levels shows the decisive fact: the seeded initial condition is *equally unequal* for the working $N = 50$ run and both failing $N = 100$ runs, but the relaxation equalises the *bulk* in every case and, at $N = 100$, does so by *sacrificing a single cell*. The runt is therefore **manufactured during**

relaxation, at the level where cells condense into compact blobs, and only at high N; it is not inherited from the initial condition, and a finer mesh deepens rather than heals it.

1 Question

Phase 1 (the Γ-convergence relaxation, `src/optimization/pgd_optimizer.py`) optimises a density field $u \in \mathbb{R}^{V\times N}$, $u_k(x) \in [0,1]$ ("how strongly cell k claims vertex x"), minimising

$$E(u) = \varepsilon \sum_k u_k^\top K_i\, u_k + \tfrac{1}{\varepsilon} \sum_k \big(u_k^2(1-u_k)^2\big)^\top M \big(u_k^2(1-u_k)^2\big) + \lambda\, P(u),$$

subject to sum-to-one ($\sum_k u_k(x) = 1$) and **equal areas** ($\int u_k\, dA = A/N$ for every cell), both enforced exactly at every step by `src/optimization/projection.py`. The interface width is tied to the mesh, $\varepsilon = \sqrt{\text{mean triangle area}} \approx h$.

The partition that Phase 2 consumes is not the density field but its *winner-take-all* classification: each vertex is awarded to the cell of largest density, and a cell's *territory* is the area of the vertices it wins. Nothing in E or the constraints references the argmax, so two different "areas" coexist: the *continuous mass* $\int u_k\, dA$ (held at A/N exactly) and the *discrete territory* (uncontrolled). For a healthy cell (density ≈ 1 on a compact blob) they agree. The question: *why, at $N = 100$, do they diverge so badly that Phase 2 fails — and where does that divergence originate?*

2 Method

The discrete territory of a solution is the winner-take-all lumped-mass area: with v the lumped P1 mass per vertex and $w_i = \arg\max_k u_k(x_i)$, cell k's territory is $a_k = \sum_{i:\, w_i = k} v_i$. We compute a_k for (i) the final solution of each run (from `solution/`) and (ii) the end-of-level density snapshot at each of the five refinement levels (from each run's `traces/` HDF5), rebuilding the per-level mesh at its resolution with `src/surfaces/torus.py`. Cell identity is preserved across levels (the inter-level interpolation keeps the column index), so a single cell can be tracked level to level. Two summaries per snapshot: the *spread* ($\mathrm{std}(a_k)/(A/N)$) and the *worst-cell error* ($\max_k |a_k - A/N|/(A/N)$).

Anchor. The worst-cell *absolute* deviation $\max_k |a_k - A/N|$ equals the Phase 2 equal-area constraint violation at iteration 0 (the two are the same quantity). For the finer $N = 100$ run this reconstruction gives 0.160, matching the 1.60e−1 logged by Phase 2 — confirming the mesh rebuild and density reshape are correct before any conclusion is drawn.

3 Results

3.1 The failure and its seed

Both $N = 100$ runs failed Phase 2: the perimeter rose every iteration and IPOPT stopped with "converged to a point of local infeasibility." The cause is a grossly infeasible *start*: the worst cell is far from the equal-area target, so the feasibility-first solver spends the run fighting the area imbalance — growing the undersized cell pushes boundaries outward, which lengthens perimeter. Table 1 contrasts the two failures with the clean $N = 50$ run.

	N=50 (works)	N=100 coarse	N=100 finer
finest mesh (V)	348×328 (114,144)	348×328 (114,144)	494×464 (229,216)
final ε	0.010186	0.010186	0.007188
worst-cell area deviation	**0.8%** (0.0036)	22.5% (0.053)	**67.3%** (0.160)
Phase 2 perimeter	152→**130** (**−14.5**%)	217→229 (rises)	216→226 (rises)
Phase 2 outcome	converges (viol → 10^{-11})	infeasibility crash	infeasibility crash

Table 1: The Phase 2 failure is seeded by the Phase 1 partition. $N = 50$ and $N = 100$-coarse ran on the *identical* finest mesh, so the driver is N, not the mesh; the finer mesh has a *sharper* ε yet a *worse* runt.

3.2 The runt cell

In the finer run, cell 92 holds its full continuous mass ($\int u_{92}\, dA = 0.2369 = A/N$ exactly) but wins only 33% of it as territory (0.0774); the other 67% is a diffuse low halo over its neighbours' ground (Fig. 1). It is *not* dormant — peak density 1.0, wins 689 vertices — merely tiny on the map.

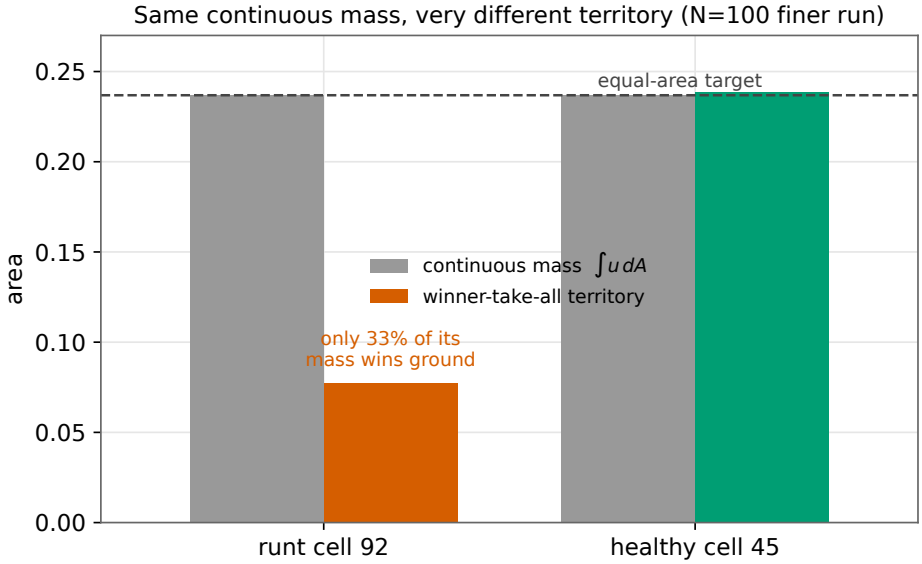

Figure 1: Same continuous mass, very different territory. The runt (cell 92) and a healthy cell hold the same paint ($\int u\, dA = A/N$, grey), but the runt's winner-take-all territory is a third of target because two-thirds of its mass is diffuse halo.

3.3 Where the runt is born

Tracking every cell's territory across the five levels (Fig. 2, Table 2) gives the central result. The *spread* collapses for all three runs (left panel) — the relaxation equalises the bulk everywhere, at a "condensation" level (L1 for $N = 50$, L2 for $N = 100$) where cells sharpen into compact blobs. But the *worst cell* (right panel) tells the opposite story at $N = 100$: while the bulk tightens, one cell is left behind and *deepens*.

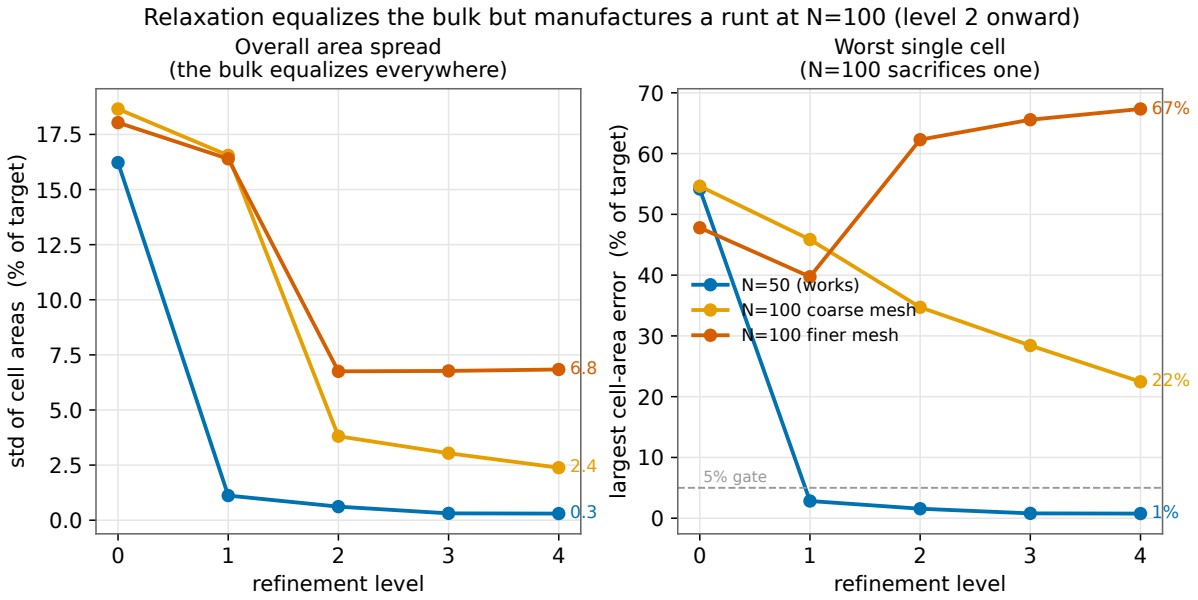

Figure 2: Across refinement levels. *Left:* the standard deviation of cell areas falls for every run — the bulk equalises. *Right:* the single worst cell — $N = 50$ collapses below 1%, the $N = 100$ coarse run stalls near 22%, the $N = 100$ finer run *deepens* to 67%.

worst-cell error (% of target)	L0 (init)	L1	L2	L3	L4 (final)
N=50	54.2	2.8	1.6	0.8	**0.8**
N=100 coarse	54.6	45.9	34.7	28.4	**22.5**
N=100 finer	47.8	39.7	62.3	65.6	**67.3**

Table 2: Worst-cell area error by level. All three seeded inits are badly unequal ($\sim 50\%$); $N = 50$ recovers fully by L1, both $N = 100$ runs do not — the finer run's runt is *created* at L2 (from -27% to -62% for cell 92) and only worsens.

Three facts follow. (1) The seeded init is **equally unequal for all three runs** (spread 16–19%, worst cell $\sim 50\%$) — so the init is *not* the discriminator. (2) The relaxation equalises the **bulk** in every run. (3) At $N = 100$ it equalises the bulk by **sacrificing one cell**, which absorbs the residual imbalance — permitted because the equal-*continuous*-mass constraint is satisfied by parking the lost territory as halo. The sacrificed cell is not the init's worst cell (cell 92 was middling at -27% at L0; the initially-worst cell recovers), so the runt cannot be predicted from the starting point.

3.4 The final distribution

The end state makes the "single sacrifice" explicit (Fig. 3): at $N = 100$ exactly one cell is a catastrophic outlier (-22% coarse, -67% finer) while the other 99 sit within a few percent of target.

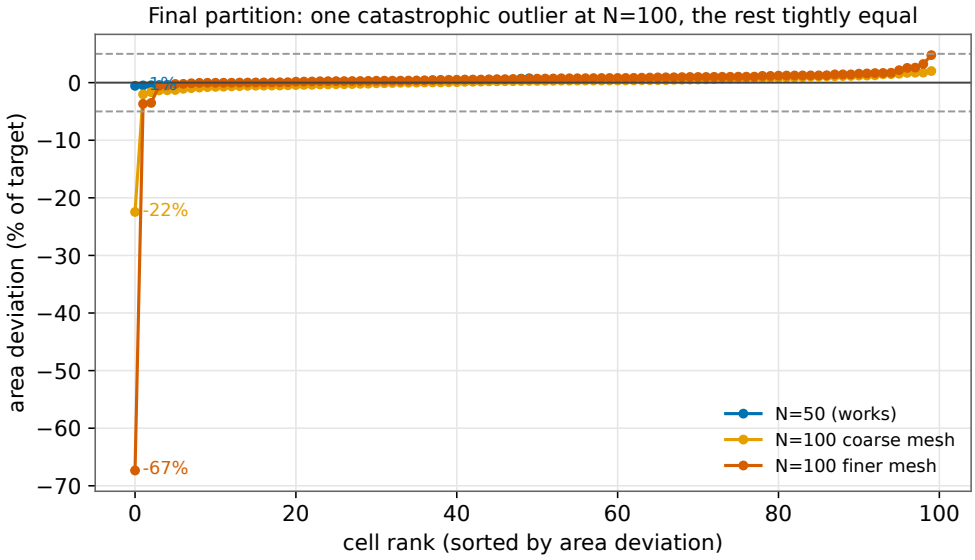

Figure 3: Final per-cell area deviation, sorted. One catastrophic outlier at $N = 100$; every other cell is within $\pm 5\%$. $N = 50$ is uniformly tight.

4 Conclusion

The high-N Phase 2 failure is a Phase 1 partition-quality problem, and the runt is **manufactured during relaxation**, at the condensation level, and only at high N — not inherited from the initial condition. The driver is N (cells shrink toward the mesh-tied interface width ε), not the mesh: $N = 50$ and $N = 100$-coarse share the identical finest mesh, and the finer mesh — with a *sharper* ε — produces a *deeper* runt (67% vs. 22%), the loser of a more decisive condensation. A finer mesh is thus the wrong lever.

5 Implications for a fix

Because the relaxation *actively* manufactures the runt — nothing forbids a cell from parking its mass as halo — the fix must change what the optimiser is *required* to achieve, not merely its starting point. This reorders the candidates (full ranking in the reference doc `winner_take_all_partition_gap.md`): an **additional constraint** on discrete territory (a hard crispness floor, or annealed soft-territory equality) is the front-runner; **improving the initial condition is downgraded** (the working $N = 50$ run starts equally unequal); soft λ tuning is insufficient; a seed re-roll is a non-scaling stopgap. The cheapest next step is to prototype the crispness floor on a fast proxy and check whether the level-2 sacrifice disappears, before spending a ~ 40 h $N = 100$ run.